



دانشگاه صنعتی شریف

دانشکده مهندسی کامپیوتر

دوره کارشناسی ارشد

زمان تحویل تمرین: ۰۷ مرداد ۱۴۰۵

تاریخ انتشار: ۱۰ تیر ۱۴۰۵

مدرس: سولماز سلیمی

## تمرین سوم: تحلیل پویا و کشف خطای حافظه با JTAG

### موضوع تمرین

در این تمرین با یک ثابت افزار آزمایشگاهی کار می کنید که برای بردهای خانوادگی ESP32 نوشته شده است. ثابت افزار چند وظیفه‌ی هم زمان دارد، از راه UART فرمان می گیرد، چند وضعیت داخلی را در RAM نگه می دارد و در نگاه اول مانند یک دستگاه ساده و سالم رفتار می کند. با این حال، در مسیر پردازش برخی فرمان ها یک خطای حافظه‌ی عمدی و کنترل شده قرار داده شده است. این خطا در شرایط خاص، بدون آنکه فرمان مستقیمی برای تغییر وضعیت داده شده باشد، یکی از مقدارهای داخلی دستگاه را تغییر می دهد.

هدف تمرین این است که این تغییر ناخواسته را فقط با حدس و آزمون ساده توضیح ندهید، بلکه با JTAG، OpenOCD و GDB مسیر اجرای برنامه را دنبال کنید، مقدارهای حافظه را در زمان اجرای برنامه ببینید، روی متغیر حساس نقطه‌ی پایش بگذارید و با شواهد واقعی نشان دهید خطا دقیقاً از کدام تابع و کدام مسیر اجرایی ایجاد شده است. در پایان نیز باید کد را اصلاح کنید و نشان دهید همان ورودی ها دیگر باعث تغییر ناخواسته نمی شوند.

این تمرین فقط برای برد و ثابت افزار آزمایشگاهی همین درس طراحی شده است. اتصال JTAG، خواندن حافظه یا عیب یابی روی دستگاه هایی که برای این تمرین در اختیار شما قرار نگرفته اند مجاز نیست. در این تمرین هیچ نیازی به تغییر دائمی Fuse ها، غیرفعال کردن واقعی JTAG یا دستکاری تنظیمات برگشت ناپذیر برد وجود ندارد.

### سناریوی تمرین

ثابت افزار داده شده چند وظیفه‌ی ساده دارد. نام این وظیفه ها در کد به صورت انگلیسی آمده است، اما نقش آن ها چنین است:

- وظیفه‌ی دریافت فرمان از UART، که خطاهای ورودی را می خواند و فرمان ها را جدا می کند.

- وظیفه‌ی نمونه‌برداری، که یک مقدار ساختگی حسگر را به‌روزرسانی می‌کند.
- وظیفه‌ی کنترل، که وضعیت کاری دستگاه را نگه می‌دارد.
- وظیفه‌ی پایش سلامت، که شمارنده‌ها و نشانه‌های سلامت را بررسی می‌کند.

نام‌های زیر در کد وجود دارند و برای عیب‌یابی استفاده می‌شوند:

```
uart_task
sensor_task
control_task
health_monitor_task
```

چند مقدار داخلی نیز در حافظه نگه داشته می‌شود. این مقادیر همگی از طریق خروجی UART دیده نمی‌شوند و برای مشاهده‌ی دقیق آن‌ها باید از GDB استفاده کنید. نمونه‌ی این متغیرها چنین است:

```
static volatile uint32_t g_safety_cookie = 0xCE40876A;
static volatile int g_device_mode = 0;
static volatile int g_safety_level = 3;
static volatile uint32_t g_event_counter = 0;
static volatile uint32_t g_health_flags = 0;
```

ثابت‌افزار از UART فرمان‌های زیر را می‌پذیرد:

```
help
status
sample
setmode normal
setmode service
stress <n>
log
heartbeat
```

فرمان **status** فقط نمای بیرونی وضعیت دستگاه را نشان می‌دهد. در این تمرین باید نشان دهید تفاوت میان آنچه روی UART دیده می‌شود و آنچه واقعاً در حافظه وجود دارد، چگونه با ابزار عیب‌یابی آشکار می‌شود.

## فایل‌های داده‌شده

به همراه صورت تمرین، یک پروژه‌ی ESP-IDF و چند ابزار کمکی داده می‌شود. ساختار پوشه‌ی تمرین به شکل زیر است:

```
E3/
  student_starter/
    CMakeLists.txt
    sdkconfig.defaults
```

```

main/
  CMakeLists.txt
  main.c
tools/
  openocd_esp32c3_usb_jtag.sh
  openocd_esp32s3_usb_jtag.sh
  gdb_esp32c3.sh
  gdb_esp32s3.sh
  watchpoint_demo.gdb
  serial_stress.py

```

فایل `main.c` ثابت‌افزار اولیه را پیاده‌سازی می‌کند. ابزار `serial_stress.py` برای فرستادن ورودی‌های طولانی یا تکراری به UART است. فایل `watchpoint_demo.gdb` چند دستور نمونه برای شروع کار با GDB دارد. این فایل فقط راهنماست و جای تحلیل شما را نمی‌گیرد.

## پیش‌نیازها

برای انجام تمرین به موارد زیر نیاز دارید:

- یک برد مبتنی بر ESP32-C3 یا ESP32-S3. استفاده از ESP32-C3 پیشنهاد می‌شود.
- نصب ESP-IDF به همراه OpenOCD و GDB مربوط به همان نسخه.
- کابل USB مناسب که انتقال داده را پشتیبانی کند.
- پایشگر سریال برای ارسال فرمان‌ها و دیدن خروجی ثابت‌افزار.
- آشنایی مقدماتی با زبان C، حافظه، پشته، وظیفه‌های FreeRTOS، نقطه‌ی توقف و نقطه‌ی پایش.

در بسیاری از بردهای ESP32-C3، رابط USB Serial/JTAG داخلی از همان پورت USB در دسترس است. در این حالت معمولاً به ابزار عیب‌یابی خارجی نیاز ندارید. روی ESP32-C3، پین‌های `GPIO18` و `GPIO19` برای USB D- و USB D+ استفاده می‌شوند؛ بنابراین در کد تمرین نباید این دو پایه را به عنوان GPIO معمولی به کار ببرید.

## ساخت و بارگذاری ثابت‌افزار

ابتدا وارد پوشه‌ی پروژه شوید و هدف ESP-IDF را متناسب با برد خود انتخاب کنید. برای ESP32-C3:

```

cd student_starter
idf.py set-target esp32c3
idf.py build
idf.py -p /dev/ttyACM0 flash monitor

```

برای ESP32-S3:

```
cd student_starter
idf.py set-target esp32s3
idf.py build
idf.py -p /dev/ttyACM0 flash monitor
```

اگر نام درگاه سریال در سیستم شما متفاوت است، به جای `/dev/ttyACM0` همان درگاه را وارد کنید. پس از راه اندازی، ثابت افزار باید پیام شروع، شناسه‌ی ساخت و فهرست کوتاه فرمان‌ها را چاپ کند.

## اتصال GDB و OpenOCD

برای ESP32-C3 با رابط USB-JTAG داخلی، معمولاً می‌توانید OpenOCD را این‌گونه اجرا کنید:

```
openocd -f board/esp32c3-builtin.cfg
```

سپس در یک ترمینال دیگر، GDB را روی فایل ELF پروژه اجرا کنید:

```
riscv32-esp-elf-gdb build/CE40876_E3_JTAG_Runtime_Forensics.elf
```

و به برد وصل شوید:

```
(gdb) target remote :3333
(gdb) monitor reset halt
(gdb) info registers
(gdb) bt
(gdb) continue
```

برای ESP32-S3، ابزار GDB و فایل پیکربندی OpenOCD متفاوت است:

```
openocd -f board/esp32s3-builtin.cfg
xtensa-esp32s3-elf-gdb build/CE40876_E3_JTAG_Runtime_Forensics.elf
```

در صورت نیاز می‌توانید از اسکریپت‌های آماده‌ی پوشه‌ی `tools` استفاده کنید. خروجی واقعی OpenOCD و GDB باید در گزارش شما آمده باشد.

برای این تمرین، هدف یادگیری عیب‌یابی، مشاهده‌ی حافظه و تحلیل خطای حافظه است. بنابراین تغییر دائمی تنظیمات امنیتی برد، سوزاندن eFuse یا غیرفعال کردن سخت‌افزاری JTAG بخشی از تمرین نیست و نباید انجام شود.

## مراحل انجام تمرین

## مرحله ۱: راه اندازی اولیه و ثبت شواهد

ثابت افزار را بسازید، روی برد بارگذاری کنید و از طریق UART فرمان های `help` و `status` را اجرا کنید. سپس OpenOCD و GDB را به برد وصل کنید. در گزارش خود این موارد را بیاورید:

- مدل دقیق برد و تراشه، مانند ESP32-C3 یا ESP32-S3.
- نسخه ی ESP-IDF.
- روش اتصال JTAG: رابط داخلی USB Serial/JTAG یا ابزار خارجی.
- بخش مهمی از خروجی OpenOCD که نشان دهد اتصال برقرار شده است.
- خروجی دستور `info registers`.
- یک نمونه ردیابی پشته با دستور `bt`.

## مرحله ۲: مشاهده ی وضعیت های پویای برنامه

در این مرحله باید چند مقدار داخلی برنامه را با GDB پیدا و مشاهده کنید. برای شروع می توانید از دستورهای زیر استفاده کنید:

```
(gdb) info variables g_  
(gdb) p/x &g_safety_cookie  
(gdb) p/x g_safety_cookie  
(gdb) p g_device_mode  
(gdb) p g_safety_level  
(gdb) x/32wx &g_safety_cookie
```

سپس از UART چند فرمان عادی مانند `status`، `sample` و `setmode normal` بفرستید. در گزارش نشان دهید کدام مقادیر تغییر کردند، کدام مقادیر ثابت ماندند، و خروجی UART چه نسبتی با مقادیرهای واقعی حافظه دارد.

## مرحله ۳: بازتولید تغییر ناخواسته

ثابت افزار طوری نوشته شده است که با برخی ورودی ها، یکی از مقادیرهای داخلی به شکل ناخواسته تغییر می کند. وظیفه ی شما این است که این رفتار را با یک دنباله ی مشخص از فرمان ها بازتولید کنید. نمونه ی شروع:

```
status  
stress 10  
status  
stress 100  
status
```

```
setmode service
status
```

همچنین می‌توانید از ابزار کمکی زیر استفاده کنید:

```
python3 tools/serial_stress.py --port /dev/ttyACM0 --count 100
```

در این مرحله فقط گفتن اینکه «برنامه خراب شد» کافی نیست. باید دنباله‌ی ورودی‌ها، خروجی قبل از خطا، خروجی بعد از خطا و مقدارهای مهم حافظه را مستند کنید.

## مرحله‌ی ۴: پیدا کردن محل خطا با نقطه‌ی پایش

مهم‌ترین بخش تمرین این است که به کمک نقطه‌ی پایش نشان دهید چه کدی مقدار حساس را تغییر می‌دهد. اگر دیدید `g_safety_level` بدون فرمان مستقیم تغییر می‌کند، می‌توانید این‌گونه شروع کنید:

```
(gdb) watch g_safety_level
(gdb) continue
```

پس از آن، همان ورودی‌هایی را که خطا را ایجاد می‌کردند از UART بفرستید. وقتی GDB متوقف شد، دستورهای زیر را اجرا کنید:

```
(gdb) bt
(gdb) info registers
(gdb) info locals
(gdb) frame 0
(gdb) list
```

در گزارش باید روشن باشد:

- نقطه‌ی پایش را روی کدام متغیر گذاشتید؟
- با کدام ورودی فعال شد؟
- ردیابی پشته کدام تابع را نشان داد؟
- تغییر مقدار مستقیم بود یا بر اثر نوشتن اشتباه در حافظه‌ی مجاور رخ داد؟
- شواهد شما برای این نتیجه‌گیری چیست؟

## مرحله ۵: تحلیل پشته، وظیفه‌ی جاری و علت خطا

بعد از فعال‌شدن نقطه‌ی پایش، باید مسیر اجرایی خطا را توضیح دهید. علاوه بر `bt`، متغیرهای محلی، پشته و وظیفه‌ی جاری را بررسی کنید. در گزارش پاسخ دهید:

- خطا در مسیر دریافت و تجزیه‌ی فرمان رخ داده است یا در وظیفه‌ی دیگری؟
- بافر، طول ورودی، اندیس آرایه یا تبدیل نوع چه نقشی در خطا دارد؟
- چرا چنین خطایی در یک سامانه‌ی نهفته مهم است؟
- اگر این وضعیت برای تصمیم ایمنی یا امنیتی دستگاه استفاده می‌شد، پیامد آن چه بود؟

## مرحله ۶: اثر توقف پردازنده روی سامانه‌ی بی‌درنگ

در ثابت‌افزار، یک تپش سلامت و چند شمارنده‌ی ساده وجود دارد. وقتی با GDB پردازنده را متوقف می‌کنید، گذر زمان از نگاه برنامه و ابزار عیب‌یابی یکسان نیست. هدف این بخش این است که ببینید خود ابزار عیب‌یابی چگونه می‌تواند رفتار یک سامانه‌ی بی‌درنگ را تغییر دهد.

یک آزمایش پیشنهادی:

```
(gdb) monitor reset halt
(gdb) continue
# wait a few seconds, then send status over UART
(gdb) interrupt
# keep target halted for 5 to 10 seconds
(gdb) continue
# send status over UART again
```

در گزارش توضیح دهید:

- آیا مقدار `event_counter`، `health_flags` یا تپش سلامت پس از توقف تغییر کرد؟
- آیا توقف پردازنده باعث هشدار یا پایان‌مهلت شد؟
- چرا عیب‌یابی یک سامانه‌ی بی‌درنگ می‌تواند نتیجه‌ی آزمایش را تغییر دهد؟
- هنگام تحلیل خطاهای پویای برنامه، این نکته چه محدودیتی ایجاد می‌کند؟

## مرحله ۷: اصلاح کد و آزمون پس از اصلاح

پس از پیدا کردن علت خطا، کد را اصلاح کنید. اصلاح شما باید متناسب با علت واقعی خطا باشد. برای نمونه ممکن است لازم باشد یکی از موارد زیر را انجام دهید:

- بررسی طول ورودی پیش از کپی کردن آن در بافر.
- استفاده از تابع امن‌تر برای کپی رشته.
- بررسی محدوده‌ی اندیس آرایه.
- جدا کردن وضعیت‌های حساس از بافرهای ورودی.
- اعتبارسنجی نوع و محدوده‌ی مقدارهای فرمان.

پس از اصلاح، همان ورودی‌هایی را که قبلاً خطا ایجاد می‌کردند دوباره اجرا کنید. باید نشان دهید وضعیت حساس دیگر ناخواسته تغییر نمی‌کند یا نقطه‌ی پایش دیگر مسیر خطا دار قبلی را گزارش نمی‌دهد.

## مرحله ۸: تحلیل امنیتی و مقاوم‌سازی

در پایان، یک تحلیل کوتاه ولی دقیق بنویسید و به پرسش‌های زیر پاسخ دهید:

- چرا JTAG در زمان توسعه مفید است اما در محصول نهایی می‌تواند خطرناک باشد؟
- تفاوت نسخه‌ی ساخت عیب‌یابی و نسخه‌ی ساخت انتشار از نظر اطلاعات قابل مشاهده چیست؟
- چرا رمزنگاری و احراز اصالت، جایگزین ایمنی حافظه نمی‌شوند؟
- اگر رمزگذاری حافظه‌ی فلش فعال باشد، آیا مشاهده‌ی RAM با ابزار عیب‌یابی هنوز مسئله است؟ چرا؟
- اگر راه‌اندازی امن فعال باشد، باز بودن رابط عیب‌یابی چه ریسکی ایجاد می‌کند؟
- برای محصول واقعی چه سیاستی برای JTAG، لاگ‌ها، نمادهای برنامه و نسخه‌های ساخت عیب‌یابی پیشنهاد می‌کنید؟

## تست‌های پیشنهادی

در گزارش خود حداقل این موارد را پوشش دهید:



| سناریو                                            | خروجی مورد انتظار                           |
|---------------------------------------------------|---------------------------------------------|
| اتصال OpenOCD و GDB                               | برنامه قابل توقف و ادامه دادن باشد          |
| خواندن ثبات‌ها و ردیابی پشته                      | خروجی واقعی در گزارش ثبت شود                |
| اجرای <code>status</code> و مشاهده‌ی وضعیت با GDB | مقدارهای UART و RAM مقایسه شوند             |
| اجرای ورودی‌های <code>stress</code>               | تغییر ناخواسته بازتولید شود                 |
| استفاده از نقطه‌ی پایش                            | محل تغییر وضعیت مشخص شود                    |
| تحلیل وظیفه و پشته                                | مسیر اجرایی خطا توضیح داده شود              |
| آزمون پس از اصلاح                                 | تغییر ناخواسته دیگر رخ ندهد یا کنترل شود    |
| توقف پردازنده با ابزار عیب‌یابی                   | اثر روی تپش سلامت یا پایشگر سلامت تحلیل شود |

## تحويل دادنی‌ها

**نکته‌ی مهم:** تمام تحويل دادنی‌ها باید در سامانه‌ی CW بارگذاری شوند. ارسال از طریق پیام‌رسان یا ایمیل برای این تمرین پذیرفته نیست.

### ۱. کد کامل پروژه

پروژه‌ی ESP-IDF اصلاح‌شده و همه‌ی فایل‌هایی که برای ساخت و اجرای آن لازم است.

### ۲. گزارش فنی

شامل مدل برد، نسخه‌ی ESP-IDF، روش اتصال JTAG، دنباله‌ی ورودی‌ها، علت خطا، شواهد GDB و توضیح اصلاح انجام‌شده.

### ۳. رونوشت واقعی از GDB و OpenOCD

شامل دستورهای مهم مانند `target remote`، `monitor reset halt`، `info registers`، `bt`، `watch` و خروجی آن‌ها.

### ۴. خروجی UART

شامل فرمان‌های ارسال‌شده و وضعیت دستگاه پیش و پس از بازتولید خطا.

### ۵. شواهد سخت‌افزاری

عکس اتصال برد یا توضیح دقیق اینکه از رابط داخلی USB Serial/JTAG استفاده کرده‌اید یا ابزار خارجی.

### ۶. ویدئو یا اسکرین‌کست کوتاه

حدود ۵ تا ۸ دقیقه که راه‌اندازی JTAG، خواندن وضعیت، فعال‌شدن نقطه‌ی پایش و آزمون پس از اصلاح را نشان دهد.

## نمره دهی (۲۰ نمره)

| نمره | مورد ارزیابی                                         |
|------|------------------------------------------------------|
| ۳    | راه اندازی JTAG، OpenOCD و GDB با شواهد واقعی        |
| ۳    | خواندن ثبات‌ها، ردیابی پشته و وضعیت‌های پویای برنامه |
| ۲    | بازتولید دقیق تغییر ناخواسته با ورودی مشخص           |
| ۳    | استفاده‌ی درست از نقطه‌ی توقف یا نقطه‌ی پایش         |
| ۳    | پیدا کردن مسیر ایجادکننده‌ی خطای حافظه و تحلیل علت   |
| ۲    | اصلاح کد و آزمون پس از اصلاح                         |
| ۲    | تحلیل اثر توقف پردازنده روی سامانه‌ی بی‌درنگ         |
| ۲    | گزارش، رونوشت، ویدئو و تکرارپذیری                    |

## راهنمای کوتاه خطاهای رایج

- اگر OpenOCD برد را پیدا نمی‌کند، مطمئن شوید کابل USB شما داده را هم منتقل می‌کند و فقط کابل شارژ نیست.
- اگر از ESP32-C3 استفاده می‌کنید، بررسی کنید برد شما واقعاً رابط داخلی USB Serial/JTAG را به پورت USB وصل کرده باشد.
- روی ESP32-C3 از `GPIO18` و `GPIO19` به عنوان GPIO معمولی استفاده نکنید، چون برای USB D- و USB D+ استفاده می‌شوند.
- اگر GDB به `3333`: وصل نمی‌شود، ابتدا مطمئن شوید OpenOCD هنوز در ترمینال دیگر در حال اجرا است.
- اگر نمادهای برنامه در GDB دیده نمی‌شوند، مطمئن شوید GDB را با فایل درست ELF از پوشه‌ی `build` اجرا کرده‌اید.
- اگر نقطه‌ی پایش فعال نمی‌شود، ابتدا مطمئن شوید متغیر درست را با دستور `watch` پایش کرده‌اید و دنباله‌ی ورودی واقعاً خطا را بازتولید می‌کند.
- تعداد نقطه‌های پایش سخت‌افزاری محدود است؛ هم‌زمان نقطه‌های پایش زیاد نگذارید.
- اگر پس از توقف طولانی، رفتار برنامه تغییر کرد، این الزاماً خطای کد نیست؛ ممکن است اثر توقف پردازنده روی سامانه‌ی بی‌درنگ باشد و باید در گزارش تحلیل شود.
- اگر برنامه پس از عیب‌یابی پاسخ نمی‌دهد، یک بار `monitor reset halt` یا بازنشانی سخت‌افزاری را امتحان کنید.
- برای این تمرین هیچ eFuse دائمی را تغییر ندهید.

## ایمیل دستیاران آموزشی

| نام دستیار آموزشی | آدرس ایمیل                           |
|-------------------|--------------------------------------|
| سپهر ابراهیمی     | sepehr.ebrahimi@ce.sharif.edu        |
| امین انصاری       | mohammadamin.ansarijafari@sharif.edu |
| مهدی بحرینی       | bahreiny@ce.sharif.edu               |